

# День 10 • 🎯 Мультиагентные системы на LangGraph, архитектуры Swarm и Supervisor

## 📋 Обсуждаемые темы:

- 🤖 Что такое мультиагентная архитектура и зачем она нужна
- 🧱 Базовые концепции и блоки библиотеки LangGraph
- 🧩 Основные паттерны: Subagents, Handoffs, Router, Skills
- 🏗️ Архитектуры: Swarm, Supervisor и Иерархические команды

## **Мультиагентная архитектура: главная идея**

Мультиагентные системы – подход к созданию интеллектуальных систем, где несколько автономных агентов взаимодействуют между собой для решения сложных задач.

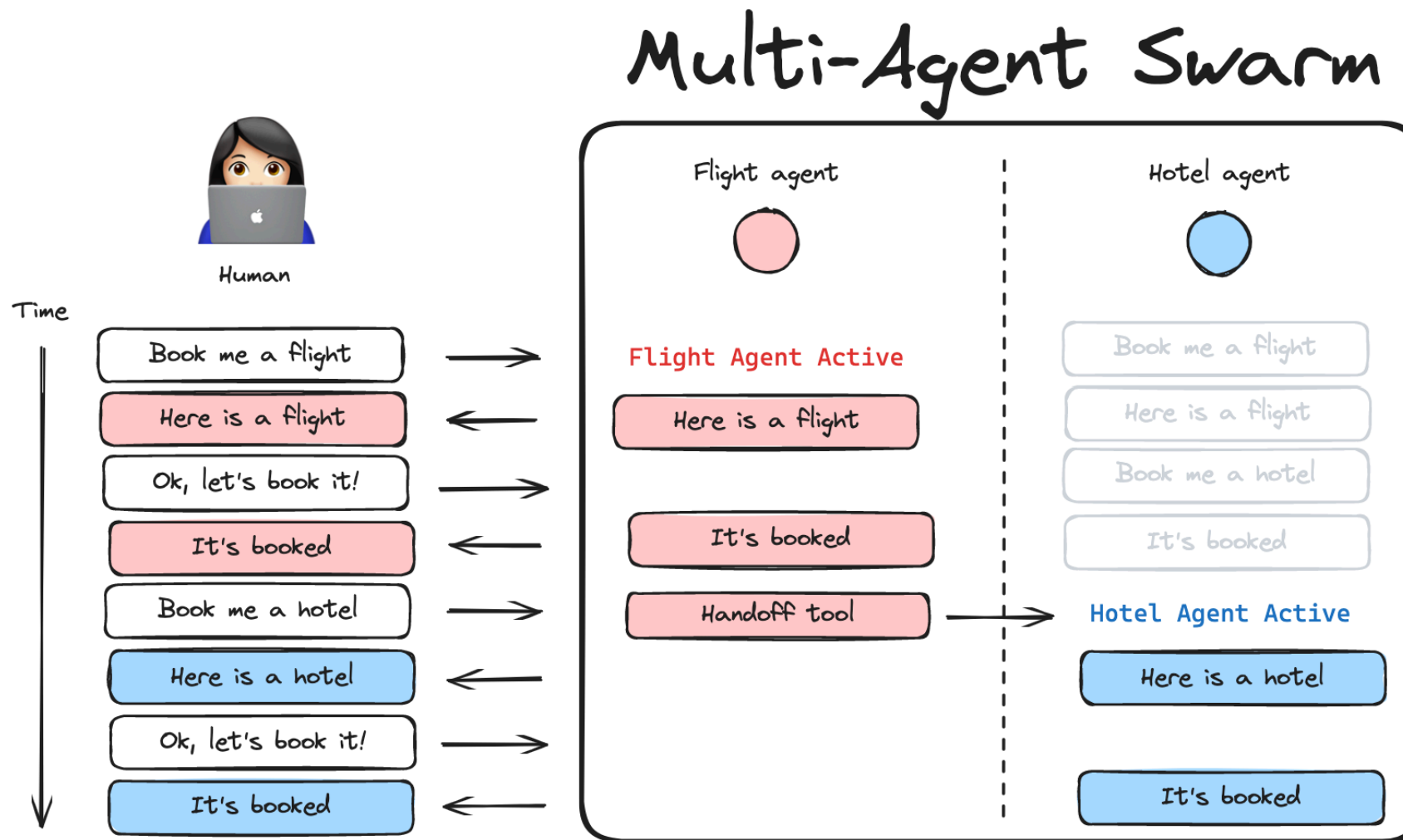
 **Ключевая идея:** Разделение сложной задачи на подзадачи, каждую из которых решает специализированный агент.

**Зачем это нужно?**

-  **Управление контекстом:** Подача специализированных знаний без переполнения контекстного окна LLM.
-  **Распределенная разработка:** Разные команды могут независимо создавать и поддерживать своих агентов.
-  **Параллелизация:** Запуск воркеров для одновременного выполнения независимых подзадач.



# Мультиагентная архитектура: в картинках (реализация Swarm)



## Концепция библиотеки LangGraph

LangGraph – фреймворк для создания *stateful*, мультиагентных приложений на основе графов.

В отличие от базового LangChain, LangGraph дает точный контроль над логикой переходов и памятью (состоянием) на каждом шаге.

### Основные строительные блоки:

-  **State (Состояние):** Разделяемая память (например, словарь), передаваемая между узлами. Хранит контекст (запрос, промежуточные ответы).
-  **Nodes (Узлы):** Шаги выполнения. Обычно это Python-функции (агенты или инструменты), которые читают и обновляют State.
-  **Edges (Рёбра):** Связи между узлами. Определяют логику переходов (последовательные или условные — Conditional Edges).



## Мультиагентная архитектура: пример кода

```
# 🛠 Пример графа с роутером (Router)
from langgraph.graph import StateGraph, START, END

# 📊 Создаём граф состояний
workflow = StateGraph(AgentState)

# 🔍 Добавляем узлы (агентов)
workflow.add_node("router_agent", router_agent)
workflow.add_node("search_agent", search_agent)
workflow.add_node("math_agent", math_agent)

# 🔗 Создаём логику переходов (условные и прямые рёбра)
workflow.add_edge(START, "router_agent")
# Логика выбора следующего узла на основе запроса:
workflow.add_conditional_edges("router_agent", routing_logic)
workflow.add_edge("search_agent", END)
workflow.add_edge("math_agent", END)

# Сборка приложения
app = workflow.compile()
```

## Концепция агента в LangGraph

Агенты – автономные сущности, способные:

-  Воспринимать данные и хранить контекст (State).
-  Обработать информацию с помощью LLM.
-  Принимать решения и вызывать инструменты (Tools).

 Основные типы агентов:

-  **ReAct Agents (Reason + Act):** Способны размышлять и динамически использовать инструменты в цикле.
-  **Chain-of-Thought:** Подробно объясняют логику решения без использования инструментов.
-  **Custom Agents:** Произвольные функции, реализующие любую логику маршрутизации и обработки состояний.

Паттерны проектирования для разных сценариев:

1.  **Subagents (Сабагенты):** Главный агент координирует подагентов, вызывая их как инструменты. Весь роутинг проходит через центр. Идеально для параллелизма и больших контекстов.
2.  **Handoffs (Передача управления):** Агенты передают контроль друг другу. Поведение меняется динамически на основе обновленного состояния. Экономит вызовы LLM при повторяющихся задачах.
3.  **Router (Маршрутизатор):** Специальный шаг классифицирует запрос и направляет его профильным агентам. Результаты затем синтезируются.
4.  **Skills (Навыки):** Один агент загружает специализированные промпты и знания "на лету" (on-demand), оставаясь главным.

## Архитектура Swarm: децентрализованная работа

Swarm (Collaboration) – рой агентов, которые могут иметь общую "доску" (shared scratchpad) для общения.

-  Каждый агент видит действия других агентов в общем контексте.
-  Результаты могут дополнять друг друга, агрегироваться или оцениваться мета-агентом.
-  Используется для задач, требующих широкого охвата: мозговые штурмы, дебаты, написание текста с одновременным ревью.

### Примеры применения:

-  Генерация текста и его критика (Writer < > Critique loop).
-  Анализ документа с разных углов зрения (юридический, технический, финансовый).



## **Архитектура Supervisor: иерархия и контроль**

Agent Supervisor – агент управляет остальными, но их контексты изолированы.

-  Менеджер получает задачу, делит ее и маршрутизирует индивидуальным агентам.
-  У подагентов **своя независимая память**, в глобальную память (State) возвращается только финальный результат их работы.
-  Применяется, когда важна последовательность, контроль качества и узкая специализация (каждому агенту нужен свой специфичный промпт и инструменты).

### **Примеры применения:**

-  Менеджер по контенту: распределяет задачи на исследователя (researcher) и писателя (writer).
-  Система поэтапной обработки заказов.



## Иерархические команды агентов (Hierarchical Teams)

Расширение идеи Supervisor для очень масштабных проектов.

- 🌟 **Суть:** В качестве узлов (nodes) выступают не просто агенты, а **целые другие графы LangGraph** (субграфы).
- 👤 Топ-менеджер управляет менеджерами команд, а те управляют своими агентами.
- 📁 **Пример:** Создание IT-продукта. Топ-супервизор управляет графом **Дизайн-команды**, графом **Разработки** и графом **QA**.

## vs Swarm и Supervisor: сравнение архитектур

| Характеристика                                                                                      |  Swarm (Collaboration) |  Supervisor / Subagents |
|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
|  Структура         | Децентрализованная                                                                                      | Иерархическая (Звезда)                                                                                     |
|  Память            | Общая (Shared scratchpad)                                                                               | Изолированная (Independent)                                                                                |
|  Назначение        | Множественные решения, креатив                                                                          | Чёткая структура, делегирование                                                                            |
|  Применение        | Мозговые штурмы, ревью                                                                                  | Проектная работа, пайплайны                                                                                |
|  Затраты токенов | Высокие (из-за общей памяти)                                                                            | Оптимизированные                                                                                           |

## Мультиагентные системы на LangGraph: выводы

Мультиагентные системы на LangGraph позволяют:

-  Строить сложные, предсказуемые пайплайны с явным контролем состояний.
-  Выбирать паттерны (Router, Handoffs) и архитектуру (Swarm, Supervisor) под конкретную задачу.
-  Избегать разрастания промптов и "галлюцинаций", разделяя инструменты между специализированными агентами.

 Подход работает особенно эффективно, когда задачи требуют:

-  Делегирования и узкой специализации.
-  Оценки результатов и циклических доработок (критика).
-  Управления сложным состоянием между шагами.



## Практические рекомендации:

-  **Начинайте с простого:** Сначала решите задачу одним агентом с нужными инструментами. Переходите к мультиагентности только когда один агент перестает справляться.
-  **Тестируйте изолированно:** Проверяйте работу каждого агента и графа отдельно перед объединением в сложный иерархический пайплайн.
-  **Проектируйте состояние (State):** Грамотно спроектированная структура `State` — залог надежной передачи данных между узлами.
-  **Документируйте агентов:** Пишите четкие docstrings для инструментов и системные промпты, чтобы LLM-роутеры правильно направляли задачи.

Вы получили новые инструменты — но опыт начинается с практики... 

 Исследуйте — пробуйте код, экспериментируйте, ошибайтесь

 Углубляйтесь — каждая тема открывает новые возможности

 Создавайте — даже маленький проект лучше теории

Это невозможно, пока вы это не сделаете!

Успехов! 